



.DiMMA

Segurança Cibernética Declarativa

Documentação Oficial Completa — v1.0.0

Node.js / Express • Python / Flask

120

Testes Reais

2

Motores

0

CVEs Conhecidas

14

Camadas de Proteção

Diamantino Mitendo Massaqui

Setembro 2026

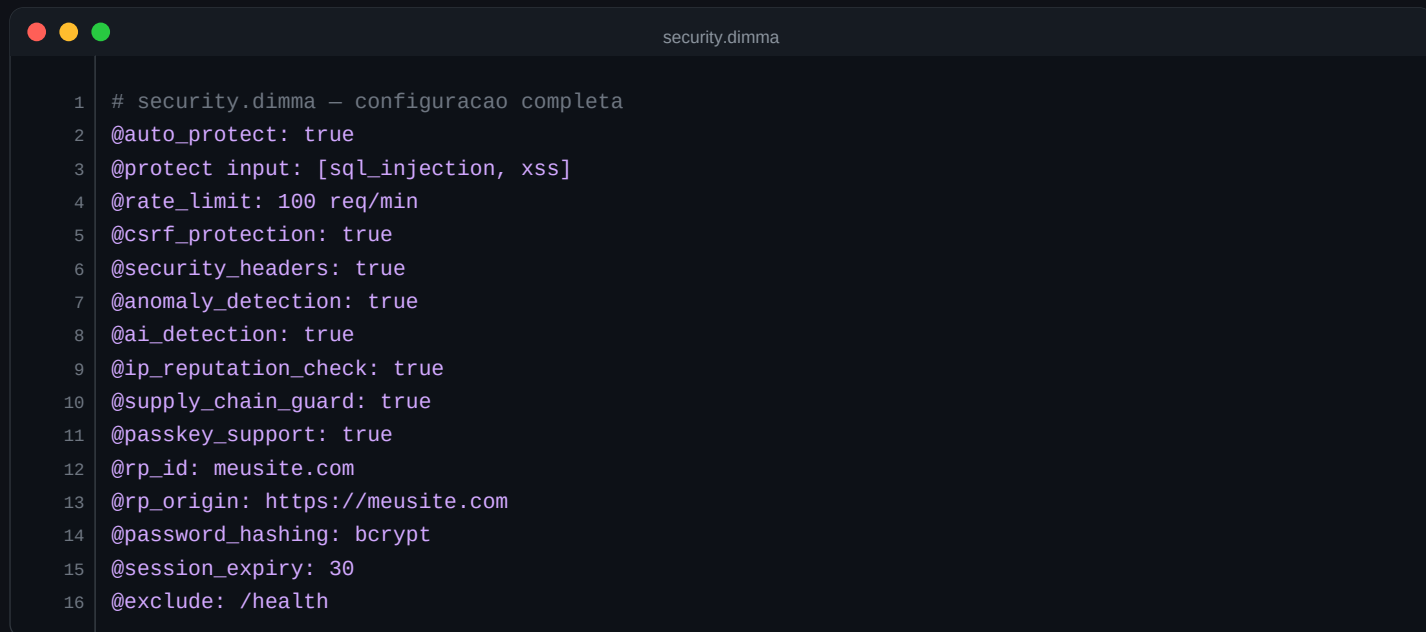
Sumário

1. O que é o .dimma?
2. Instalação
3. Sintaxe da Linguagem
4. Referência Completa de Comandos
 - @auto_protect
 - @protect input
 - @rate_limit
 - @csrf_protection
 - @security_headers
 - @anomaly_detection
 - @ai_detection — NVIDIA NIM (gratuito)
 - @ip_reputation_check — AbuseIPDB (gratuito)
 - @supply_chain_guard ★ NOVO — O mais importante de 2026
 - @passkey_support / @rp_id / @rp_origin
 - @password_hashing / @session_expiry / @exclude / @connect
5. Comandos CLI (dimma init · scan · pqc-check)
6. Integração Node.js / Express (completo)
7. Integração Python / Flask (completo)
8. CSRF no Frontend — Vanilla JS, React, Angular
9. Redis e Escalabilidade Distribuída
10. Segurança de Produção — Checklist
11. ★ Pesquisa 2026 — Supply Chain Attacks (dados reais assustadores)
12. Limitações Honestas
13. O que o .dimma NÃO substitui
14. FAQ — Perguntas Frequentes
15. Glossário

1. O que é o .dimma?

O `.dimma` é um arquivo de configuração declarativo para segurança web. Com sintaxe própria, descreve as regras de segurança do projeto num único lugar, e um motor (Node.js ou Python) aplica automaticamente as proteções — sem que o programador configure cada biblioteca manualmente.

- Pense no `.dimma` como as "regras de segurança" do projeto, escritas num único arquivo — do mesmo jeito que o `.env` guarda variáveis de ambiente. O MESMO arquivo `.dimma` funciona em Node.js e Python sem qualquer alteração.



```
1 # security.dimma — configuracao completa
2 @auto_protect: true
3 @protect input: [sql_injection, xss]
4 @rate_limit: 100 req/min
5 @csrf_protection: true
6 @security_headers: true
7 @anomaly_detection: true
8 @ai_detection: true
9 @ip_reputation_check: true
10 @supply_chain_guard: true
11 @passkey_support: true
12 @rp_id: meusite.com
13 @rp_origin: https://meusite.com
14 @password_hashing: bcrypt
15 @session_expiry: 30
16 @exclude: /health
```

2. Instalação

Node.js / Express

```
Terminal

1 npm install dimma-core
2 npx dimma init # cria o security.dimma automaticamente
3
4 # Variaveis de ambiente (.env — nunca commitar):
5 # DIMMA_CSRF_SECRET=... openssl rand -hex 32
6 # NVIDIA_API_KEY=nvapi- gratuito em build.nvidia.com
7 # ABUSEIPDB_API_KEY=... gratuito em abuseipdb.com
8 # REDIS_URL=redis://... opcional, para multiplas instancias
9 # NODE_ENV=production
```

Python / Flask

```
Terminal

1 pip install dimma
2 pip install dimma[redis] # com suporte a Redis
3 python -m dimma.cli init # cria o security.dimma
4
5 # Variaveis de ambiente:
6 # SECRET_KEY=... python -c "import secrets; print(secrets.token_hex(32))"
7 # NVIDIA_API_KEY=nvapi-...
8 # ABUSEIPDB_API_KEY=...
9 # REDIS_URL=redis://...
10 # FLASK_ENV=production
```

✗ Em PRODUÇÃO, o motor RECUSA iniciar sem DIMMA_CSRF_SECRET (Node) ou SECRET_KEY (Python). Em desenvolvimento gera um segredo aleatório temporário com aviso no console.

3. Sintaxe da Linguagem .dimma

Sintaxe

```
1 # Comentario — qualquer linha com # e ignorada
2
3 @nome_do_comando: valor           # booleano, numero ou texto
4 @comando com espacos: valor       # nomes podem ter espacos
5 @lista: [item1, item2, item3]     # valor em lista
6 @rate_limit: 100 req/min         # expressao de taxa especial
```

Tipo	Sintaxe	Exemplo	Resultado
Booleano	true/false	@auto_protect: true	true / True
Lista	[a,b,c]	@protect input: [sql_injection,xss]	["sql_injection","xss"]
Taxa	N req/unidade	@rate_limit: 100 req/min	{max:100, window_ms:60000}
Número	dígitos	@session_expiry: 30	30
Texto	string	@password_hashing: bcrypt	"bcrypt"

- Comandos de lista (@protect input, @exclude, @connect) se COMBINAM quando repetidos — os valores são unidos, não sobrescritos.

4. Referência Completa de Comandos

Comando	Descrição
@auto_protect: true	Ativa o perfil padrão com uma linha. Aplica: headers, CSRF, SQLi/XSS.
@protect input: [sql_injection, xss]	Bloqueia padrões maliciosos no body/query/params ANTES da rota.
@rate_limit: 100 req/min	Limita requisições por IP. Com Redis, compartilhado entre instâncias.
@override rate_limit: 50 req/min	Alias explícito de @rate_limit (para sobrescrever @auto_protect).
@csrf_protection: true	Proteção CSRF double-submit cookie. Requer buscar token em GET /csrf-token.
@security_headers: true	Headers HTTP seguros: CSP, HSTS, X-Frame-Options, X-Content-Type-Options.
@anomaly_detection: true	Detector estatístico (z-score). Sinaliza rajadas fora do padrão por IP.
@ai_detection: true	NVIDIA NIM (gratuito). Revisão de IA para distinguir ataque real de falso positivo.
@ip_reputation_check: true	AbuseIPDB (gratuito). Verifica histórico de abuso do IP em anomalias.
@supply_chain_guard: true	★ NOVO. Detecta pacotes maliciosos/suspeitos/alucinados por IA nas dependências.
@passkey_support: true	Passkeys/WebAuthn. Autenticação sem senha via biometria ou chave física.
@rp_id: meusite.com	Relying Party ID para WebAuthn (domínio da aplicação).
@rp_origin: https://meusite.com	Origem esperada para WebAuthn (HTTPS obrigatório).
@password_hashing: bcrypt	Hash de senha bcrypt custo 12. Expõe hashPassword/verifyPassword.
@session_expiry: 30	Sessão expira após N minutos de inatividade (configuração exposta para a app).
@exclude: /health	Rota isenta de todas as checagens. Pode ser repetido — valores se combinam.
@connect: routes/api.js	Metadado informativo — arquivo que este .dimma governa.

■ LIMITAÇÃO HONESTA: @protect input é baseado em padrões (regex). Não substitui queries parametrizadas/ORM — esses continuam obrigatórios no seu código de banco de dados.

■ @ai_detection usa NVIDIA NIM (gratuita em build.nvidia.com — sem cartão de crédito).
@ip_reputation_check usa AbuseIPDB (gratuito em abuseipdb.com — ~1000 consultas/dia). Ambas com circuit breaker diário para evitar exaustão de cota.

✗ ★ @supply_chain_guard — A PROTEÇÃO MAIS IMPORTANTE DE 2026

Ataques à cadeia de dependências aumentaram 4.5x em 2026. 1.700+ pacotes maliciosos rastreados até hackers norte-coreanos (abril 2026). O axios (83M downloads/semana) foi comprometido em março 2026. E o "slopsquatting": atacantes registram em HORAS os nomes de pacotes alucinados por IAs como Copilot e Cursor.

Verificação	Sev.	Descrição
Pacote recém-publicado (< 7 dias)	ALTO	Pacotes novos ainda não auditados pela comunidade
Maintainer sem histórico	ALTO	Conta criada recentemente para publicar um único pacote — padrão de ataque
Slopsquatting / typosquatting	CRÍTICO	Nome alucinado por IA registrado por atacante horas depois
Postinstall script suspeito	MÉDIO	Script executado automaticamente no install — vetor do ataque ao axios
CVE conhecida (audit)	CRÍTICO	Vulnerabilidade publicada nas bases oficiais npm audit / pip-audit
Hash de integridade ausente	ALTO	Entrada no lockfile sem campo integrity — bypass silencioso de verificação

GitHub Actions

```
1 # CI/CD — bloquear deploy se supply chain guard falhar
2 # .github/workflows/security.yml
3
4 jobs:
5   dimma-scan:
6     steps:
7       - run: npm install dimma-core
8       - run: npx dimma scan # exit 1 se CRITICO ou ALTO
9       # Supply chain guard verifica package.json automaticamente
```

- "Slopsquatting" — o ataque mais assustador de 2026:
1. Atacante pergunta ao Copilot/Cursor como fazer X em código
 2. A IA alucina um nome de pacote que não existe
 3. Atacante registra ESSE NOME no npm/PyPI com código malicioso
 4. Próximo dev que perguntar a mesma coisa recebe o mesmo nome
 5. Roda npm install — e está infectado
- Estudo USENIX 2025: 19.7% dos pacotes recomendados por 16 modelos de IA não existiam.

5. Comandos CLI

dimma init

Cria o security.dimma com perfil padrão, detectando automaticamente a stack do projeto.

```
Terminal
1 npx dimma init          # Node.js
2 python -m dimma.cli init # Python
3
4 # Saída:
5 # security.dimma criado com sucesso.
6 # Stack detectada: node
7 # Proximo passo: npm install dimma-core
```

dimma scan — 14 regras de detecção

Scanner estático de código inseguro. Exit code 1 em achados CRÍTICO/ALTO.

ID da Regra	Sev.	Detecta
hardcoded-aws-key	CRÍTICO	Chave AWS (AKIA...) hardcoded
hardcoded-secret-assignment	CRÍTICO	API key/senha/token em string literal
hardcoded-private-key	CRÍTICO	Chave privada RSA/EC embutida no código
sql-string-concat-js/py	CRÍTICO	SQL por concatenação de string
sql-fstring-py	CRÍTICO	SQL por f-string Python
eval-usage-js/py	ALTO	eval()/exec() com entrada dinâmica
command-injection-node	ALTO	child_process.exec() direto ou desestruturado
command-injection-py	ALTO	subprocess shell=True ou os.system()
insecure-deserialization-py	CRÍTICO	pickle.loads() ou yaml.load() sem SafeLoader
debug-mode-flask	ALTO	app.run(debug=True) ativo
innerHTML-assignment	MÉDIO	.innerHTML = dado dinâmico (XSS)
dangerously-set-innerhtml	MÉDIO	dangerouslySetInnerHTML no React
wildcard-cors	MÉDIO	CORS Access-Control-Allow-Origin: *
http-credentials-in-url	MÉDIO	usuario:senha@host embutido em URL

dimma pqc-check

Verifica suporte a criptografia pós-quântica (X25519MLKEM768, FIPS 203). Cobre conexões de SAÍDA. Para tráfego de entrada, o proxy/CDN é responsável (Cloudflare já suporta; Nginx precisa OpenSSL 3.5+).

Terminal

```
1 dimma pqc-check
2
3 # Node.js v22.20+ (OpenSSL 3.5 embutido):
4 # Status: PRONTO - X25519MLKEM768 suportado
5
6 # Python (OpenSSL do SO, geralmente desatualizado):
7 # Status: NAO PRONTO - atualize o OpenSSL do sistema
```

6. Integração Node.js / Express

JavaScript — app.js

```
1 const express = require('express');
2 const { DimmaEngine, hashPassword, verifyPassword } = require('dimma-core');
3
4 const app = express();
5 app.use(express.json());
6 // app.set('trust proxy', 1); // descomente se estiver atras de Nginx
7
8 const dimma = new DimmaEngine('./security.dimma', {
9   rpID: process.env.RP_ID,
10  origin: process.env.RP_ORIGIN,
11 });
12 dimma.protect(app); // aplica tudo do .dimma
13
14 app.get('/health', (req, res) => res.json({ status: 'ok' }));
15 app.get('/csrf-token', (req, res) =>
16   res.json({ csrfToken: dimma.generateCsrfToken(req, res) }));
17
18 app.post('/register', async (req, res) => {
19   const hash = await hashPassword(req.body.password); // bcrypt 12
20   res.json({ success: true });
21 });
22
23 app.post('/login', async (req, res) => {
24   // const ok = await verifyPassword(password, hash);
25   res.json({ message: 'Login processado' });
26 });
27
28 app.listen(3000);
```

7. Integração Python / Flask

```
Python — app.py

1 from flask import Flask, request, jsonify
2 from flask_wtf.csrf import generate_csrf
3 from dimma.engine import DimmaEngine
4 from dimma.security.auth import hash_password, verify_password
5 import os
6
7 app = Flask(__name__)
8 dimma = DimmaEngine('./security.dimma',
9     rp_id=os.environ.get('RP_ID'),
10    rp_origin=os.environ.get('RP_ORIGIN'))
11 dimma.protect(app) # aplica tudo do .dimma
12
13 @app.get('/health')
14 def health(): return jsonify(status='ok')
15
16 @app.get('/csrf-token')
17 def csrf_token(): return jsonify(csrfToken=generate_csrf())
18
19 @app.post('/register')
20 def register():
21     data = request.get_json()
22     h = hash_password(data['password']) # bcrypt 12
23     return jsonify(success=True)
24
25 @app.post('/login')
26 def login():
27     # ok = verify_password(password, stored_hash)
28     return jsonify(message='Login processado')
29
30 if __name__ == '__main__': app.run(port=5000)
```

8. CSRF no Frontend

Vanilla JS / JavaScript Puro

```
JavaScript (Vanilla)

1  const { csrfToken } = await fetch('/csrf-token').then(r => r.json());
2
3  async function securePost(url, body) {
4    return fetch(url, {
5      method: 'POST',
6      headers: { 'Content-Type': 'application/json', 'x-csrf-token': csrfToken },
7      body: JSON.stringify(body),
8      credentials: 'same-origin'
9    });
10 }
11
12 const res = await securePost('/login', { email: 'user@ex.com', password: '...' });
```

React — Hook personalizado

```
React

1  import { useState, useEffect } from 'react';
2
3  export function useDimma() {
4    const [csrfToken, setCsrfToken] = useState('');
5    useEffect(() => { fetch('/csrf-token').then(r=>r.json()).then(d=>setCsrfToken(d.csrfToken))
6
7    const securePost = (url, body) => fetch(url, {
8      method: 'POST',
9      headers: { 'Content-Type': 'application/json', 'x-csrf-token': csrfToken },
10     body: JSON.stringify(body), credentials: 'same-origin'
11   });
12
13   return { securePost };
14 }
15
16 // Uso no componente:
17 // const { securePost } = useDimma();
18 // securePost('/login', { email, password });
```

Angular — Interceptor HTTP

TypeScript (Angular)

```
1 // dimma-csrf.interceptor.ts
2 import { Injectable } from '@angular/core';
3 import { HttpInterceptor, HttpRequest, HttpHandler } from '@angular/common/http';
4 import { switchMap } from 'rxjs/operators';
5
6 @Injectable()
7 export class DimmaCsrfInterceptor implements HttpInterceptor {
8   private token: string | null = null;
9
10  intercept(req: HttpRequest<any>, next: HttpHandler) {
11    if (['POST', 'PUT', 'DELETE'].includes(req.method)) {
12      if (this.token)
13        return next.handle(req.clone({ setHeaders: { 'x-csrf-token': this.token } }));
14      return this.http.get<any>('/csrf-token').pipe(
15        switchMap(({ csrfToken }) => {
16          this.token = csrfToken;
17          return next.handle(req.clone({ setHeaders: { 'x-csrf-token': csrfToken } }));
18        }));
19    }
20    return next.handle(req);
21  }
22 }
```

9. Redis e Escalabilidade Distribuída

Sem Redis, rate limiting e detecção de anomalia ficam na memória de um único processo. Com múltiplas instâncias, um atacante distribui as requisições e nunca bate o limite. Configure REDIS_URL para proteger corretamente.

Funcionalidade	Sem Redis	Com Redis
Rate limiting	Memória local (1 proc.)	Contadores compartilhados
Anomaly detection	Histórico local	Histórico compartilhado
Circuit breaker	Cota local	Cota diária compartilhada
CSRF	Cookie stateless	Cookie stateless (não precisa Redis)

Configuração

```
1 # .env
2 REDIS_URL=redis://localhost:6379/0
3
4 # Docker Compose
5 # services:
6 #   redis:
7 #     image: redis:7-alpine
8 #
9 # Railway / Render: adicionar add-on Redis e copiar a URL
```


11. ★ Pesquisa 2026 — Supply Chain Attacks

■ Esta secção apresenta dados reais de pesquisa que fundamentam o @supply_chain_guard como a funcionalidade mais importante e extraordinária do projeto em 2026.

Dado Real de 2026	Fonte	Por que assusta
Ataques supply chain: +4.5x em 2026 vs 2025	PkgPulse Security Report	Maior aceleração já registrada neste vetor de ataque
1.700+ pacotes npm/PyPI maliciosos (abr/2026)	Socket Security/Checkmarx	Grupo norte-coreano Sapphire Sleet numa única campanha
Axios comprometido: 83M downloads/semana	DEV Community / npmjs.com	RAT instalado via conta de maintainer sequestrada
19.7% dos pacotes recomendados por IA não existem	USENIX Security 2025 (576K amostras)	Alvo perfeito para slopsquatting automático
Atacantes registram nome alucinado em horas	Socket/Aikido Security	Todo projeto que usa IA pra codar está em risco
react-codeshift: propagado por 47 AI Agent Skills	Aikido Security (jan/2026)	Infraestrutura de automação amplifica o alcance
CVE-2026-50021: pnpm bypass de lockfile integrity	OSV.dev	Lockfile correto NÃO garante instalação segura

O ataque ao axios de março 2026 demonstrou algo crítico: o hash SHA-512 do lockfile funcionou exatamente como projetado — e ainda assim o trojan foi instalado. Por quê? Porque o registro computou o hash correto para o tarball malicioso. Um npm ci encontrou correspondência e reportou "clean install" enquanto instalava o trojan.

✗ CONCLUSÃO: o lockfile não protege contra pacote malicioso publicado ANTES do seu lockfile ser criado. A defesa real é verificar comportamento e histórico do pacote ANTES de instalá-lo — exatamente o que o @supply_chain_guard faz.

12. Limitações Honestas

Limitação	O que significa	Como mitigar
Sanitização por regex	Ofuscação avançada pode passar	Use sempre prepared statements/ORM
Anomaly detection	Pode ter falsos positivos	Use como alerta, não como único bloqueio
dimma scan	Falsos positivos em strings de teste	Code review humano complementar
WebAuthn	Exige navegador real para teste completo	Testar manualmente com autenticador
PQC no Python	Depende do OpenSSL do sistema	Node.js tem OpenSSL 3.5+ embutido
Estado sem Redis	Memória local por processo	Configure REDIS_URL em produção

13. O que o .dimma NÃO substitui

O que NÃO substitui	Por quê
Certificado TLS/HTTPS	O .dimma configura headers corretos, mas o certificado é criado no servidor/proxy
Queries parametrizadas / ORM	Sanitização é camada extra. Prepared statements continuam obrigatórios
Auditoria profissional	Antes de usar com dados sensíveis reais em produção
Firewall de rede / WAF	Opera na camada de aplicação — complementa, não substitui proteção de rede
Backups e recuperação	Segurança em repouso e plano de recuperação são da infraestrutura

✓

O .dimma usa apenas bibliotecas vetadas (Helmet, bcrypt, Flask-Talisman, @simplewebauthn/server, csrf-csrf, Flask-WTF). 120 testes reais passando. 0 CVEs nas dependências (npm audit + pip-audit, setembro 2026).

14. FAQ — Perguntas Frequentes

P: A biblioteca só funciona em localhost?

R: Não. O .dimma roda onde quer que o seu servidor esteja — localhost, VPS, Docker, cloud. O localhost foi apenas o ambiente de desenvolvimento.

P: Preciso de cartão de crédito para a IA (NVIDIA NIM) ou AbuseIPDB?

R: Não. Ambas têm free tier sem cartão de crédito. NVIDIA NIM: ~1000 créditos em build.nvidia.com. AbuseIPDB: ~1000 consultas/dia em abuseipdb.com.

P: O que acontece se a API NVIDIA ou AbuseIPDB ficar fora do ar?

R: O motor falha de forma segura (fail-safe): regista o erro no console e continua processando a requisição sem o passo de IA/reputação. A aplicação nunca cai.

P: Preciso de Redis desde o primeiro dia?

R: Não. Redis só é necessário ao escalar para mais de uma instância do servidor. Sem Redis, o motor usa memória local automaticamente (com aviso).

P: Como publicar a biblioteca para outros usarem?

R: Node: npm publish (conta grátis em npmjs.com). Python: python -m build && twine upload dist/* (conta grátis em pypi.org).

P: O .dimma substitui um WAF?

R: Não. O .dimma opera na camada de aplicação. Um WAF opera na camada de rede, antes do tráfego chegar. São complementares, não concorrentes.

P: Por que usar NVIDIA NIM em vez da API da Anthropic?

R: A NVIDIA NIM é completamente gratuita (sem cartão de crédito), enquanto a Anthropic é paga. Como a camada de IA só é chamada em anomalias raras, o free tier da NVIDIA é mais do que suficiente na prática.

15. Glossário

CSRF — Cross-Site Request Forgery — site malicioso faz o browser da vítima enviar requisições autenticadas a outro site sem permissão.

XSS — Cross-Site Scripting — injeção de código malicioso em páginas web, executado no browser de outros usuários.

SQL Injection — Inserção de código SQL malicioso em campos de entrada para manipular queries do banco de dados.

Supply Chain Attack — Ataque que compromete o projeto através das suas dependências (bibliotecas de terceiros).

Slopsquatting — Atacante registra nomes de pacotes alucinados por IAs de programação (Copilot, Cursor, etc.) com código malicioso.

Rate Limiting — Limitação de quantas requisições um cliente pode fazer num período de tempo.

Anomaly Detection — Identificação de padrões de tráfego que desviam significativamente do comportamento normal.

Passkey/WebAuthn — Autenticação sem senha usando biometria ou chaves físicas. A chave privada nunca sai do dispositivo.

HSTS — HTTP Strict Transport Security — force sempre HTTPS para um domínio.

CSP — Content Security Policy — define quais fontes de conteúdo o browser pode carregar (previne XSS).

bcrypt — Algoritmo de hashing de senha com fator de custo ajustável, resistente a força bruta.

PQC — Post-Quantum Cryptography — criptografia resistente a computadores quânticos. Grupo X25519MLKEM768.

Circuit Breaker — Interrompe chamadas a serviços externos quando um limite é atingido, evitando sobrecarga.

Fail-Safe — Design onde uma falha resulta em comportamento seguro — a aplicação continua sem a feature que falhou.

Double-Submit Cookie — Padrão CSRF onde o token está no cookie E no header, e o servidor verifica que os dois batem.



.DiMMA

Segurança Cibernética Declarativa

Criado por **Diamantino Mitendo Massaqui**

```
npm install dimma-core
```

```
pip install dimma
```

[DIMMA_SYNTAX.md](#) • [DIMMA_SINTAXE.md](#) • [GUIA_DO_CURSO.md](#) • [DIMMA_PARA_LEIGOS.md](#)

v1.0.0 — Setembro 2026 — MIT License